Topic	Questions	Max Marks	Stars
IR Basics	Q1, Q2, Q3	8	★★
TF and IDF	Q4	6	—
Vector Space Model	Q5, Q6	9	★★★
NER Basics + Metrics	Q7, Q8	8	★★
Entity + Relation Extraction	Q9–Q12	9	★★
Building NER System	Q13, Q14	9	—
Coreference Resolution	Q15–Q18	8	★
Compare all three tasks	Q19	8	—
CLIR	Q[illegible]23	9	★★★

1) **Describe the concept of Information Retrieval system in Natural Language Processing. [4 Marks] ★★ OR Explain Information Retrieval architecture with neat diagram. [8 Marks] OR What is Information Retrieval, and how is it used in NLP? [2 Marks]**

Information Retrieval (IR) is the process of finding relevant documents or information from a large collection of text based on a user's query.

Simple analogy: When you type something on **Google**, it searches through billions of web pages and returns the most relevant results in seconds. That is Information Retrieval in action. You give a query — the system finds and ranks the best matching documents for you.

IR answers the question: *"Given this query, which documents are most relevant?"*

IR in NLP — How They Connect

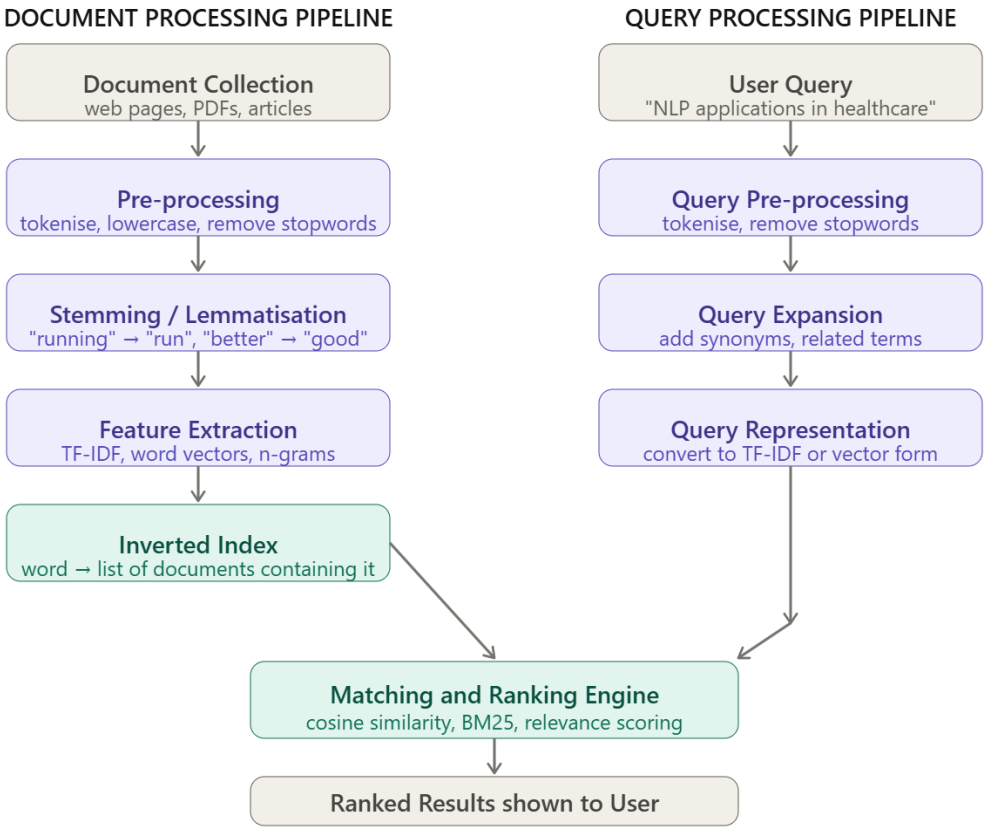
IR and NLP are deeply connected. NLP techniques are used inside IR systems to better understand both the query and the documents.

NLP Technique	How it helps IR
Tokenisation	Breaks text into words for indexing
Stemming / Lemmatisation	Matches "running" with "run"
Stop word removal	Removes "the", "is", "a" to reduce noise
Named Entity Recognition	Identifies important names and places in queries
TF-IDF	Ranks documents by word importance

NLP Technique	How it helps IR
Word embeddings	Finds semantically similar documents

Key Components of an IR System---

IR Architecture



A. Document Collection

The set of all documents the IR system will search through. These can be web pages, research papers, news articles, PDFs, or any text content.

B. Pre-processing

Raw documents are cleaned before indexing:

- **Tokenisation** — split text into individual words
- **Lowercasing** — "NLP" and "nlp" treated as same
- **Stop word removal** — remove "the", "is", "and" (not useful for search)
- **Stemming** — reduce words to their root form ("running" → "run")

C. Feature Extraction

Documents are converted into numerical form using:

- **TF-IDF vectors** — weight words by importance
- **Bag of Words** — count word frequencies
- **Word embeddings** — semantic representation

D. Inverted Index

The most important data structure in IR. It maps each word to the list of documents that contain it — like the index at the back of a textbook.

"NLP" → [Doc1, Doc3, Doc7, Doc12] "healthcare" → [Doc3, Doc5, Doc9]

This allows very fast lookup — instead of searching every document, just look up the word in the index.

E. Query Processing

When a user types a query:

1. Same pre-processing as documents (tokenise, remove stopwords)
2. **Query expansion** — add synonyms to improve recall
3. Convert query to same vector format as documents

F. Matching and Ranking

Compare query vector with all document vectors using **cosine similarity:**

Similarity = (Q · D) / (|Q| × |D|)

Documents are ranked from most to least similar. Top-k documents are returned to the user.

Applications of IR in NLP

- **Search engines** — Google, Bing rank web pages for queries
- **Question answering** — find documents containing the answer
- **Document summarisation** — retrieve key sentences from a document
- **Plagiarism detection** — find similar documents in a corpus
- **Recommendation systems** — suggest similar articles or products

and stemming to TF-IDF weighting and semantic matching. IR is the foundation of modern search engines, question answering systems, and recommendation platforms.

2) **Define Term Frequency (TF) and Inverse Document Frequency (IDF) with respect to Information Retrieval. [6 Marks]**

In Information Retrieval, when a user searches for something, the system must decide **which documents are most relevant** to the query. Simply counting how many times a word appears is not enough — because common words like "the", "is", "a" appear everywhere and are not useful.

TF and IDF together solve this problem by giving **high importance to words that are frequent in one specific document but rare across all documents.**

Simple analogy: In a library of medical books, the word **"the"** appears in every book — it is useless for finding relevant books. But the word **"chemotherapy"** appears only in cancer-related books — it is very informative. TF-IDF automatically identifies and weights such informative words higher.

Term Frequency (TF)

Term Frequency measures how often a term appears in **one specific document.** A word that appears many times in a document is likely important to that document.

TF (t, d) = Number of times term t appears in document d / Total number of terms in document d**Example:**

Document: "cricket player scores in cricket match cricket game today" Total words = 10 Count of "cricket" = 3

TF ("cricket", D1) = 3 / 10 = 0.3

Key point about TF:

- TF only measures **local importance** — how important the word is within one document
- Problem — common words like "the", "is" also get high TF — they appear everywhere
- Solution — IDF fixes this by penalising common words

Inverse Document Frequency (IDF)

Inverse Document Frequency measures how **rare or unique** a term is across the entire document collection. If a word appears in very few documents, it is more informative and gets a higher IDF score.

IDF (t) = log (N / df(t))

Where:

- **N** = total number of documents in the collection
- **df(t)** = number of documents that contain term t
- **log** = logarithm (base 10 or natural log)**Example:**

Collection has N = 5 documents "the" appears in all 5 documents → df = 5 **IDF ("the") = log(5/5) = log(1) = 0**

"quantum" appears in only 1 document → df = 1 **IDF ("quantum") = log(5/1) = log(5) = 0.699**

TF-IDF Combined

TF and IDF are multiplied together to get the final importance score:

TF-IDF (t, d) = TF (t, d) × IDF (t)

This ensures:

- Words **frequent in one document AND rare in corpus** → high score → very important
- Words **common everywhere** → IDF = 0 → score = 0 → automatically filtered out

Worked Example in IR Context

Document collection (3 documents):

	D1	**D2**	**D3**
"cricket"	3	0	1
"the"	2	3	2
Total words	10	8	9

For "cricket" in D1:

Step	Calculation	Result
TF	3 / 10	0.300
df	appears in D1 and D3	2
IDF	log(3/2)	0.176
TF-IDF	0.300 × 0.176	**0.0528**

For "the" in D1:

Step	Calculation	Result
TF	2 / 10	0.200
df	appears in D1, D2, D3	3
IDF	log(3/3) = log(1)	**0.000**
TF-IDF	0.200 × 0.000	**0.000**

"the" is completely filtered out — exactly what we want in IR.

Role of TF-IDF in IR

In an IR system, every document is converted into a **TF-IDF vector** where:

- Each dimension = one word in vocabulary
- Each value = TF-IDF score of that word in that document

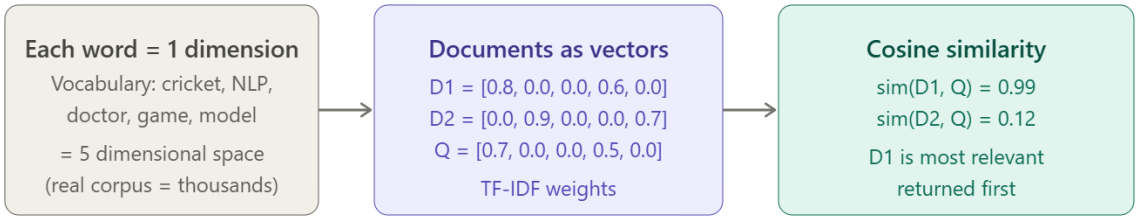
When a query comes in, it is also converted to a TF-IDF vector. The system finds documents whose TF-IDF vectors are **closest to the query vector** using cosine similarity.

3) Explain the concept of the Vector Space Model (VSM) and describe how it is used in Information Retrieval. [6 Marks] ★★★ OR Describe the Vector Space Model (VSM) for information retrieval. How does VSM represent documents and queries, and how are similarities calculated? Discuss strengths and weaknesses. [9 Marks] ★★

The **Vector Space Model (VSM)** is a mathematical model used in Information Retrieval where both **documents and queries are represented as vectors** in a multi-dimensional space. Each dimension in this space represents one unique word from the vocabulary.

Simple analogy: Imagine a 2D map where every city (document) has a GPS location. When you search for something (query), it also gets a GPS location on the same map. The cities closest to your search location are the most relevant results. VSM works exactly like this — but instead of 2 dimensions, it uses thousands of dimensions (one per word).

Core Idea---



How VSM Represents Documents

Every document is converted into a **TF-IDF vector.** Each position in the vector corresponds to one word in the vocabulary. The value at each position is the TF-IDF weight of that word in that document.

Example — Vocabulary: {cricket, NLP, doctor, game, model} → 5 dimensions

Document	**cricket**	**NLP**	**doctor**	**game**	**model**
D1 — sports article	0.80	0.00	0.00	0.60	0.00
D2 — NLP article	0.00	0.90	0.00	0.00	0.70
D3 — medical article	0.00	0.00	0.85	0.00	0.20

So:

D1 = [0.80, 0.00, 0.00, 0.60, 0.00] D2 = [0.00, 0.90, 0.00, 0.00, 0.70] D3 = [0.00, 0.00, 0.85, 0.00, 0.20]

Words with zero TF-IDF = word not present or too common → ignored automatically.

How VSM Represents Queries

The user query is processed and converted into the **exact same vector format** as documents.

Query: "cricket game" Query vector = [0.70, 0.00, 0.00, 0.50, 0.00]

The query vector has non-zero values only for the words present in the query.

How Similarity is Calculated — Cosine Similarity

Once both documents and query are represented as vectors, VSM uses **cosine similarity** to measure how close the query vector is to each document vector.

Cosine Similarity (Q, D) = (Q · D) / (|Q| × |D|)

Where:

- **Q · D** = dot product of query and document vectors
- **|Q|** = magnitude of query vector
- **|D|** = magnitude of document vector**Why cosine and not Euclidean distance?**

Cosine similarity measures the **angle** between two vectors — not the distance. This means a long detailed document and a short document about the same topic will have the **same similarity score** to the query — because they point in the same direction. Euclidean distance would unfairly penalise longer documents.

Step-by-Step Worked Example

Query: "cricket game" **Documents:**

D1: "cricket player scores in cricket game" (6 words) D2: "NLP model learns language patterns" (5 words)

Step 1 — Build TF-IDF vectors:

	cricket	**game**	**NLP**	**model**	**player**
D1	0.48	0.24	0.00	0.00	0.24
D2	0.00	0.00	0.45	0.45	0.00
Q	0.70	0.50	0.00	0.00	0.00

Step 2 — Compute cosine similarity:

sim(Q, D1):

Q · D1 = (0.70×0.48) + (0.50×0.24) + 0 + 0 + 0 = 0.336 + 0.120 = **0.456** |Q| = √(0.70² + 0.50²) = √(0.49 + 0.25) = √0.74 = **0.860** |D1| = √(0.48² + 0.24² + 0.24²) = √(0.2304 + 0.0576 + 0.0576) = √0.3456 = **0.588 sim(Q, D1) = 0.456 / (0.860 × 0.588) = 0.456 / 0.506 = 0.901**

sim(Q, D2):

Q · D2 = 0 (no common words) **sim(Q, D2) = 0.000**

Result: D1 is ranked first — highly relevant. D2 is ranked last — not relevant.

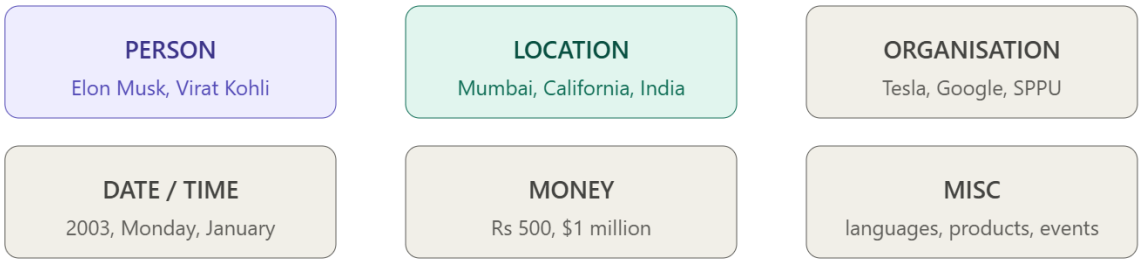
4) **What is Named Entity Recognition (NER)? Describe the various metrics used for evaluation. [8 Marks] ★★ OR What is Named Entity Recognition (NER)? [2 Marks] OR Describe the Named Entity Recognition (NER) system building process. Given a sample sentence, outline the steps to build an NER system using a supervised learning approach. [9 Marks] OR Discuss the different methods used for evaluating NER systems. What are common metrics and how can results be analyzed to improve the system? [9 Marks]**

Named Entity Recognition (NER) is an NLP task that automatically identifies and classifies **named entities** — specific real-world objects like people, places, organisations, dates, and amounts — in a given text.

Simple analogy: Imagine reading a news article and highlighting all the important names with different colour pens — yellow for person names, blue for places, green for organisations. NER does exactly this — automatically, for thousands of documents in seconds.

Input: "Elon Musk founded Tesla in California in 2003." **Output:** Elon Musk [PERSON] founded Tesla [ORG] in California [LOCATION] in 2003 [DATE]

Types of Named Entities---



How NER Works — Overview

NER treats the problem as a **sequence labelling task.** Every word in the sentence gets a label.

Most NER systems use the **BIO tagging scheme:**

Tag	Meaning
B	Beginning of an entity
I	Inside (continuation) of an entity
O	Outside — not an entity

Example:

Word	Tag	Entity
Elon	B-PERSON	Start of person name
Musk	I-PERSON	Continuation of person name
founded	O	Not an entity
Tesla	B-ORG	Start of organisation
in	O	Not an entity
California	B-LOC	Location

NER Pipeline---

Approaches Used in NER

Approach	How it works	Example
Rule-based	Hand-written rules and dictionaries	If word starts with capital → likely entity
Statistical (HMM, CRF)	Learn patterns from labelled data	CRF learns context patterns
Deep Learning (BiLSTM, BERT)	Neural networks learn features automatically	BERT-based NER

Evaluation Metrics for NER

After building an NER system, we need to measure how good it is. There are three main metrics:

Metric 1 — Precision

Precision measures — out of all entities the model **predicted**, how many were **correct?**

Precision = True Positives / (True Positives + False Positives)

Precision = Correctly identified entities / Total entities predicted by model

Example:

Model predicted 10 entities. 7 were correct. **Precision = 7 / 10 = 0.70 = 70%**

High precision = when the model says "this is an entity", it is usually right.

Metric 2 — Recall

Recall measures — out of all entities that **actually exist** in the text, how many did the model **find?**

Recall = True Positives / (True Positives + False Negatives)

Recall = Correctly identified entities / Total actual entities in text

Example:

Text actually had 9 entities. Model found 7 of them. **Recall = 7 / 9 = 0.78 = 78%**

High recall = model finds most of the entities — misses very few.

Metric 3 — F1 Score

F1 Score is the **balance between Precision and Recall.** It is used when both precision and recall are important.

F1 = 2 × Precision × Recall / (Precision + Recall)

Example:

Precision = 0.70, Recall = 0.78 F1 = 2 × 0.70 × 0.78 / (0.70 + 0.78) F1 = 1.092 / 1.48 **F1 = 0.738 = 73.8%**

Building an NER System — Supervised Learning with Sample Sentence [Practical Walkthrough]

Step 1 — Collect Training Data

Gather a large collection of raw text from relevant sources — news articles, Wikipedia pages, government documents.

For our system: collect 10,000 news sentences about Indian politics, science, and geography.

More data = better model. Minimum recommended = **1000 labelled sentences.**

Step 2 — Annotate Data using BIO Tagging

Human annotators label every word in every sentence with a BIO tag.

BIO Scheme:

Tag	Meaning
B-TYPE	Beginning of a named entity
I-TYPE	Inside / continuation of entity
O	Not a named entity

Applying BIO to our sentence:**Complete annotation of our sentence:**

Word	BIO Tag	Explanation
Narendra	B-PER	Beginning of person name
Modi	I-PER	Continuation of person name
visited	O	Not an entity — verb
ISRO	B-ORG	Beginning of organisation
headquarters	O	Not an entity — common noun
in	O	Not an entity — preposition
(... more rows below)		Bengaluru=B-LOC, 15th=B-DATE...

Step 3 — Pre-processing

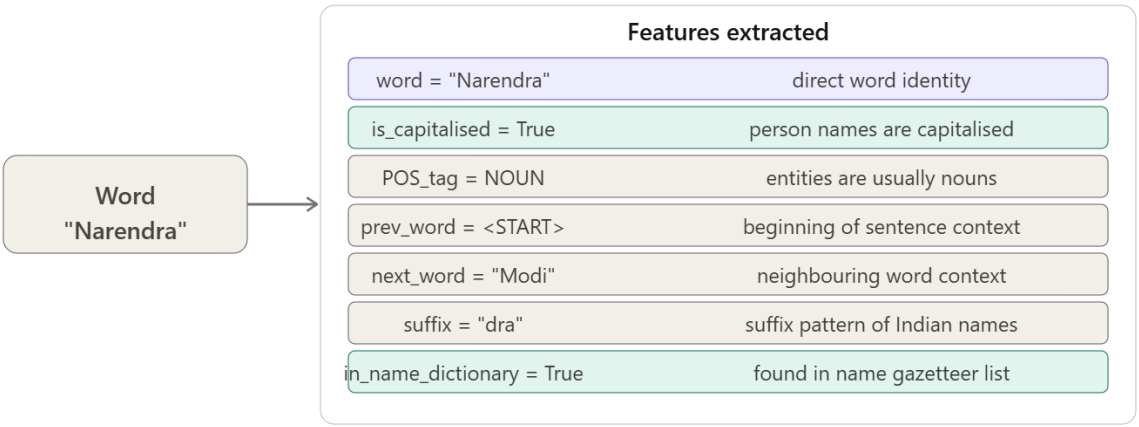
Clean and prepare the annotated data before feeding it to the model:

Pre-processing Step	What it does	Example
Tokenisation	Split text into words	"Narendra Modi" → ["Narendra", "Modi"]
Sentence splitting	Break paragraph into sentences	One sentence per line
Train-Test split	80% training, 20% testing	8000 train, 2000 test sentences
Shuffle	Randomise order	Prevents model learning position bias

Step 4 — Feature Extraction

For every word, extract a feature vector — a list of properties that help the model decide if it is an entity.

Features extracted for each word:

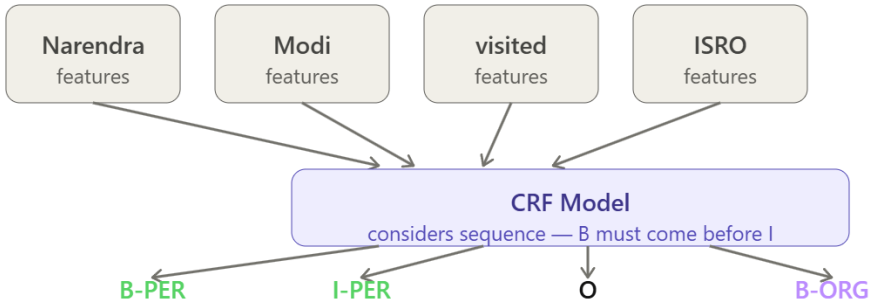


This feature vector is created for **every word** in every training sentence.

Step 5 — Train the Model

Feed all feature vectors + BIO labels into a supervised learning model. The model learns which feature combinations predict which BIO tags.

For supervised NER, CRF (Conditional Random Field) is commonly used:



Why CRF? CRF considers the **entire sequence** — it knows that after a B-PER tag, only I-PER or O is valid. It cannot output B-ORG immediately after B-PER. This sequence awareness makes NER predictions more consistent.

Step 6 — Test and Evaluate

Run the trained model on the **test set** (20% data the model never saw):

Model output on our sentence:

Word	Actual Tag	Predicted Tag	Correct?
Narendra	B-PER	B-PER	✅

Word	Actual Tag	Predicted Tag	Correct?
Modi	I-PER	I-PER	✅
visited	O	O	✅
ISRO	B-ORG	B-ORG	✅
headquarters	O	O	✅
Bengaluru	B-LOC	B-LOC	✅
15th	B-DATE	O	❌ Missed
August	I-DATE	O	❌ Missed
2024	I-DATE	B-DATE	❌ Wrong boundary

Compute metrics:

TP = 4 (Narendra Modi, ISRO, Bengaluru correctly found) FN = 1 (date entity missed) FP = 1 (2024 tagged with wrong boundary)

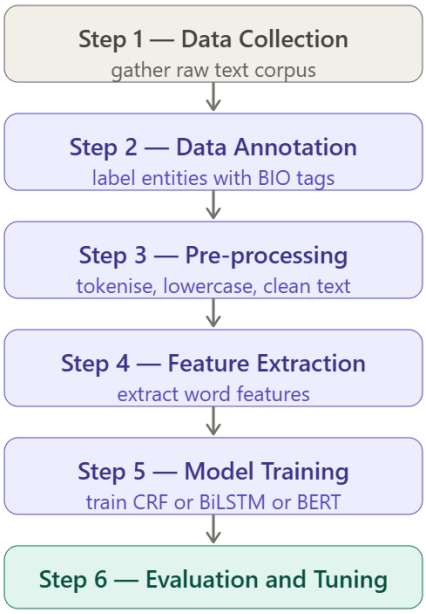
Precision = 4 / (4+1) = 0.80 Recall = 4 / (4+1) = 0.80 F1 = 0.80

Step 7 — Improve and Redeploy

From the evaluation we found DATE entities are weak. Actions to improve:

- Add more labelled sentences with date patterns
- Add date-specific features — "is this word a number?", "is next word a month name?"
- Retrain and re-evaluate until F1 is acceptable (typically > 0.85 for production)

Complete NER System Building Process



Step 1 — Data Collection

Collect a large amount of **raw text** from relevant domains — news articles, Wikipedia, medical records, legal documents depending on the use case.

For our example, we collect news articles about technology companies.

The more data collected, the better the model will learn.

Step 2 — Data Annotation (Labelling)

This is the most important step. Human annotators read through the text and **label every word** with its correct entity tag using the **BIO scheme.**

BIO Tagging:

- **B** = Beginning of entity
- **I** = Inside (continuation) of entity
- **O** = Outside — not an entity

Applying BIO to our sample sentence:

Word	BIO Tag	Entity Type
Sundar	B-PER	Beginning of person name
Pichai	I-PER	Continuation of person name
joined	O	Not an entity

Word	BIO Tag	Entity Type
Google	B-ORG	Beginning of organisation
in	O	Not an entity
Mountain	B-LOC	Beginning of location
View	I-LOC	Continuation of location
in	O	Not an entity
2004	B-DATE	Date entity
.	O	Punctuation

Step 3 — Pre-processing

Clean and prepare the annotated text for training:

- **Tokenisation** — split text into individual words and punctuation
- **Sentence splitting** — break paragraph into individual sentences
- **Lowercasing** — convert to lowercase (optional — capitalisation is often a useful feature)
- **Train-test split** — split data into 80% training and 20% testing

Step 4 — Feature Extraction

For each word, extract features that help the model decide if it is an entity:

Feature	Example for "Sundar"	Why useful
Word itself	"Sundar"	Direct word identity
Is capitalised?	Yes	Person names often start with capital
POS tag	NOUN	Most entities are nouns
Previous word	<start>	Position in sentence matters
Next word	"Pichai"	Neighbouring words give context
Word suffix	"-ar"	Suffix patterns help
Is in dictionary?	Yes (name list)	Known entity dictionaries

Feature	Example for "Sundar"	Why useful
Word shape	Xx (capital + lower)	Capitalisation pattern

Step 5 — Model Training

Train a supervised model on the labelled data. Common models used:

Option A — CRF (Conditional Random Field)

- Traditional ML approach
- Uses hand-crafted features from Step 4
- Models the **sequence context** — considers neighbouring words
- Good baseline model, fast to train

Option B — BiLSTM-CRF

- Deep learning approach
- BiLSTM reads the sentence from both left to right AND right to left
- Captures longer context than CRF
- CRF layer at output ensures valid tag sequences

Option C — BERT-based NER (best performance)

- Pre-trained BERT model fine-tuned on NER data
- Each word gets a rich contextual embedding
- Small output layer added — classifies each word's embedding as entity type

For our supervised approach — using CRF:

Training input: feature vectors for each word Training output: correct BIO tag for each word Model learns: which feature combinations indicate which entity types

Step 6 — Evaluation

Test the trained model on the held-out test set:

- Compute **Precision, Recall, F1 score** for each entity type
- Identify which entity types the model struggles with
- Fine-tune features or retrain with more data for weak categories

Step 7 — Deployment

Once the model achieves acceptable F1 score, deploy it:

- Wrap model in an API
- Accept raw text input → return annotated text with entity tags
- Monitor performance on real-world inputs and retrain periodically

5) Explain Entity Extraction and Relation Extraction with respect to NER. [6 Marks] ★ OR Describe entity extraction and relation extraction with the help of examples. [8 Marks] ★★

When we read a sentence like:

"Sachin Tendulkar played for Mumbai Indians in the IPL tournament."

As humans we automatically understand:

- **Who** — Sachin Tendulkar (a person)
- **What team** — Mumbai Indians (an organisation)
- **What event** — IPL tournament (an event)
- **What relationship** — Sachin **played for** Mumbai Indians

NLP systems do this in two stages — first **Entity Extraction**, then **Relation Extraction.**---

Part A — Entity Extraction

Entity Extraction is the process of automatically **identifying and classifying specific meaningful pieces of information** (called entities) from unstructured text.

Simple analogy: Imagine reading a newspaper and using a highlighter to mark every important name, place, organisation, date, and amount. Entity Extraction does exactly this — automatically, for any text, in seconds.

What counts as an entity?

Entity Type	Example from news
PERSON	Virat Kohli, Narendra Modi
ORGANISATION	ISRO, Infosys, BCCI
LOCATION	Mumbai, Pune, Delhi

Entity Type	Example from news
DATE / TIME	15th August, 2024
MONEY	Rs 500 crore, $1 million
PRODUCT	iPhone 15, Tesla Model 3

How Entity Extraction works — BIO Tagging:

Every word in the sentence is tagged with a BIO label:

- **B** = Beginning of entity
- **I** = Inside / continuation of entity
- **O** = Not an entity

Example sentence:

"Virat Kohli scored 100 runs for Royal Challengers Bangalore."

Word	BIO Tag	Entity
Virat	B-PER	Start of person
Kohli	I-PER	Continue person
scored	O	Not entity
100	B-NUM	Number
runs	O	Not entity
for	O	Not entity
Royal	B-ORG	Start of team
Challengers	I-ORG	Continue team
Bangalore	I-ORG	Continue team

Output of Entity Extraction:

Virat Kohli → **PERSON** 100 → **NUMBER** Royal Challengers Bangalore → **ORGANISATION**

Entity Extraction with respect to NER:

NER is a **specific type** of Entity Extraction that focuses only on standard named entities — Person, Location, Organisation, Date, Money. Entity Extraction is broader — it can also extract domain-specific entities like drug names, legal terms, or product specifications.

NER ⊂ Entity Extraction (NER is a subset of Entity Extraction)

Part B — Relation Extraction

Relation Extraction takes the entities found by Entity Extraction and **identifies the relationship between pairs of entities** in the text.

Simple analogy: Entity Extraction finds all the characters in a story. Relation Extraction finds out who is the boss, who is the friend, who works where. Finding the characters alone is not enough — the relationships between them are what create meaning.

What a relation looks like:

"Sundar Pichai is the CEO of Google."

Entities: **Sundar Pichai** [PERSON], **Google** [ORG] Relation: Sundar Pichai → **is_CEO_of** → Google

This is called a **Knowledge Triple:**

(Subject, Relation, Object) (Sundar Pichai, is_CEO_of, Google)

Common Relation Types:

Relation	Example sentence	Triple
works_for	"Rohit works at TCS"	(Rohit, works_for, TCS)
located_in	"ISRO is in Bengaluru"	(ISRO, located_in, Bengaluru)
founded_by	"Infosys was founded by Narayana Murthy"	(Infosys, founded_by, Narayana Murthy)
plays_for	"Sachin played for Mumbai"	(Sachin, plays_for, Mumbai)
treats	"Aspirin treats headache"	(Aspirin, treats, headache)

Complete Worked Example — Both Together

Input sentence:

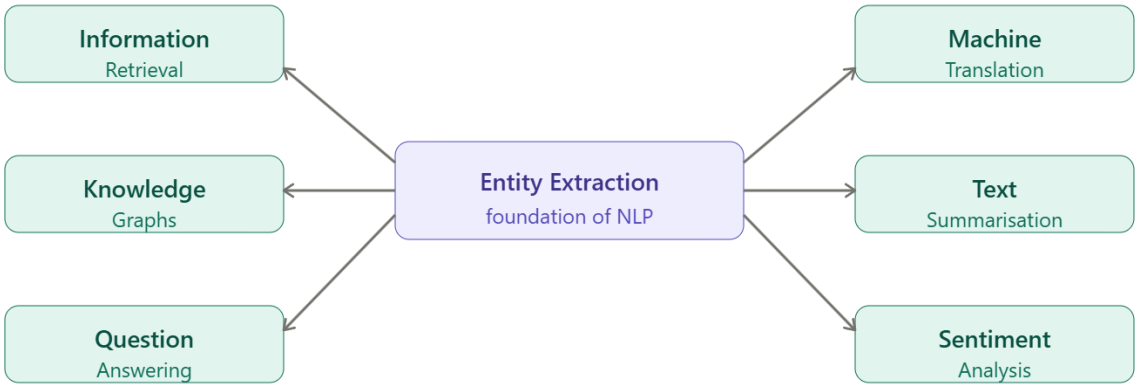
*"Ratan Tata founded Tata Motors in Mumbai in 1945 with an investment of Rs 100 crore."***Final structured output:**

Triple	Meaning
(Ratan Tata, **founded**, Tata Motors)	Ratan Tata started Tata Motors
(Tata Motors, **located_in**, Mumbai)	Company is in Mumbai
(Tata Motors, **founded_in**, 1945)	Year of founding

6) Explain the importance of entity extraction in NLP. How does entity extraction differ from named entity recognition? Provide examples of real-world applications. [9 Marks] OR Explain the process of Entity Extraction in Information Retrieval. Discuss various techniques and algorithms used. [8 Marks]

Importance of Entity Extraction in NLP

Entity extraction is one of the most fundamental tasks in NLP. Almost every advanced NLP application depends on it.



Why entity extraction is important:

- **Converts unstructured to structured data** — 80% of world's data is unstructured text. Entity extraction makes it machine-readable.
- **Enables semantic search** — search engines find pages based on meaning, not just keywords.
- **Powers knowledge graphs** — Google's Knowledge Panel ("Elon Musk — CEO of Tesla") is built using entity extraction.
- **Saves human effort** — manually reading millions of documents is impossible. Entity extraction automates this.
- **Improves downstream NLP tasks** — Question Answering, Summarisation, Relation Extraction all depend on entities being found first.

Techniques and Algorithms for Entity Extraction

Technique 1 — Rule-Based Approach

Uses hand-written patterns and dictionaries (called gazetteers) to find entities.

Pattern: word ending in "pur", "bad", "nagar" → likely LOCATION Pattern: matches DD/MM/YYYY format → DATE Gazetteer: if word is in "list of Indian cities" → LOCATION

- **Pros:** Fast, high precision, no training data needed
- **Cons:** Cannot find unseen entities, rules break for new text styles

Technique 2 — Machine Learning — CRF (Conditional Random Field)

CRF is the most widely used traditional ML approach. It:

- Takes feature vectors for each word (capitalisation, POS tag, neighbours)
- Considers the **sequence context** — knows B must come before I
- Learns patterns from labelled training data
  Features for "Bengaluru": is_capitalised=True, POS=NOUN, in_city_list=True → LOC
- **Pros:** Handles sequence, good generalisation
- **Cons:** Needs feature engineering, needs labelled data

Technique 3 — Deep Learning — BiLSTM

BiLSTM reads the sentence from **both left to right and right to left**, capturing rich context for each word.

"Apple [ORG or FRUIT?] launched iPhone" BiLSTM sees "launched iPhone" on the right → predicts ORG not FRUIT

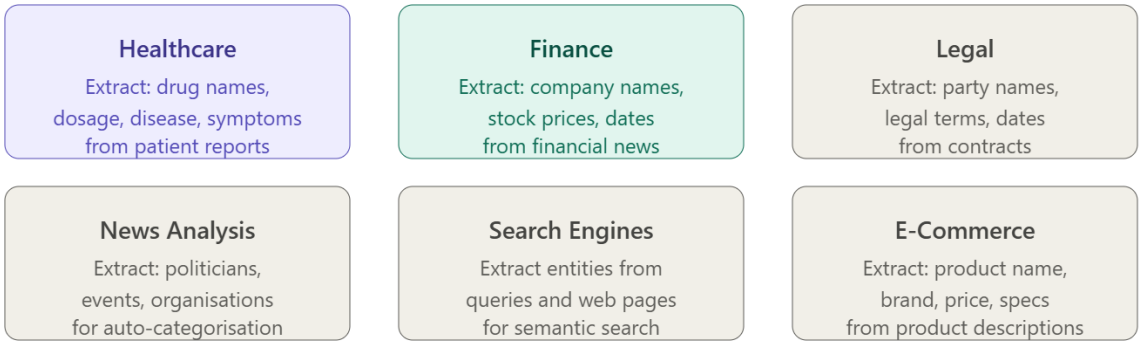
- **Pros:** Better context understanding, no feature engineering
- **Cons:** Needs large training data, slower to train

Technique 4 — Transformer-based — BERT

BERT is the most powerful approach today. It uses **pre-trained language knowledge** and is fine-tuned for entity extraction.

- Each word gets a contextual embedding
- A classification layer on top predicts the entity tag for each word
- Best performance — especially for ambiguous entities

Real-World Applications of Entity Extraction



7) **Describe co-reference resolution with example. [4 Marks] ★ OR What is Coreference Resolution? Give examples. [4 Marks] ★ OR Explain reference resolution and coreference resolution with example. [8 Marks] ★ OR Explain the concept of reference resolution and coreference resolution in NLP. How do these techniques help in understanding relationships between entities? Provide examples. [8 Marks]**

When we write or speak, we often use **pronouns and short phrases** to refer back to something already mentioned — instead of repeating the full name every time.

"Sachin Tendulkar is a great cricketer. ***He*** *scored 100 centuries.* ***The master blaster*** *retired in 2013."*

Here **"He"** and **"The master blaster"** both refer to **"Sachin Tendulkar"**. A human understands this instantly. But a computer sees three different expressions and gets confused.

Reference Resolution is the NLP task of figuring out **what each pronoun or referring expression actually refers to** in the text.

Simple analogy: Imagine reading a mystery novel where characters are sometimes called by their name, sometimes "he", sometimes "the detective", sometimes "the tall man". Reference resolution is like keeping a mental map of who each of these expressions refers to — so you never lose track of who did what.

Coreference Resolution is a specific type of reference resolution. It finds all words and phrases in a text that **refer to the same real-world entity** and groups them together.

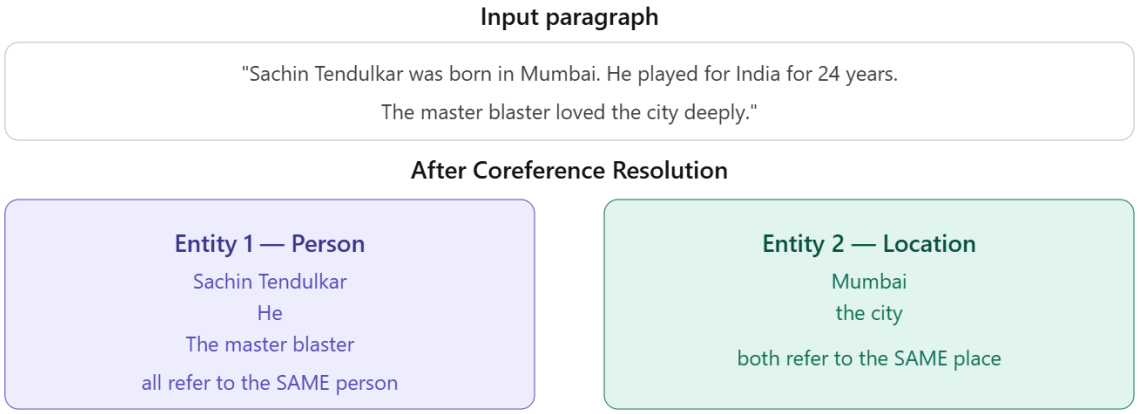
Two expressions are **coreferent** if they refer to the same thing in the real world.

Simple analogy: In a class register, students are listed by full name. But during class, the teacher calls them "you", "this student", "the boy in the back". Coreference resolution matches all these different references back to the same student in the register.

Key Terms

Term	Meaning	Example
Mention	Any noun phrase that refers to an entity	"Sachin", "He", "The batsman"
Antecedent	The first / main mention of the entity	"Sachin Tendulkar"
Coreference chain	All mentions that refer to the same entity	{Sachin Tendulkar, He, The master blaster}
Coreference cluster	The grouped set of all coreferent mentions	Same as chain

Visual Example of Coreference---



Detailed Example 1 — Simple Coreference

*"Narendra Modi visited ISRO. **He** congratulated the scientists. **The Prime Minister** called it India's greatest achievement."*

Coreference chains:

Mention	Refers to
Narendra Modi	Entity 1 — PERSON
He	Entity 1 — PERSON
The Prime Minister	Entity 1 — PERSON
ISRO	Entity 2 — ORGANISATION
it	Entity 2 — ORGANISATION

Mention	Refers to
the scientists	Entity 3 — GROUP

Result after resolution:

Narendra Modi visited ISRO. **[Narendra Modi]** congratulated the scientists. **[Narendra Modi]** called **[ISRO]** India's greatest achievement.

Now the system clearly knows who did what — no ambiguity.

How Coreference Resolution Works — Steps

Step 1 — Detect all mentions

Find every noun phrase, pronoun, and named entity that could refer to something.

Mentions in our example: {Narendra Modi, ISRO, He, the scientists, The Prime Minister, it}

Step 2 — Extract features for each mention pair For every pair of mentions, extract features that help decide if they are coreferent:

Feature	Example
Gender agreement	"Sachin... He" — both masculine ✓
Number agreement	"Scientists... they" — both plural ✓
Distance	Closer mentions more likely to corefer
Syntactic role	Subject-subject pairs more likely
Semantic similarity	"Prime Minister... Modi" — same semantic type ✓

Step 3 — Link coreferent mentions Use a classifier or neural model to decide which mention pairs are coreferent. Chain all linked mentions into clusters.

Step 4 — Output clusters

Cluster 1: {Narendra Modi, He, The Prime Minister} Cluster 2: {ISRO, it} Cluster 3: {the scientists}

Reference Resolution vs Coreference Resolution

Aspect	Reference Resolution	Coreference Resolution
Scope	Broader — any kind of reference	Specific type of reference resolution

Aspect	Reference Resolution	Coreference Resolution
What it does	Resolves what any expression refers to	Groups all expressions referring to same entity
Includes	Pronouns, definite descriptions, demonstratives	Specifically coreferent mention chains
Example	"this", "that", "the former"	"Modi... He... The PM"
Relationship	Parent concept	Subset of reference resolution

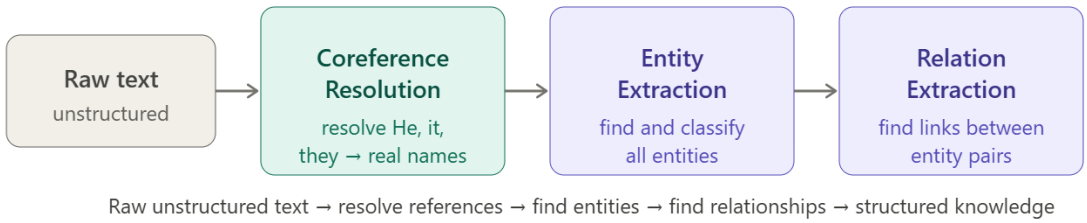
8) Compare and contrast Entity Extraction, Relation Extraction and Coreference Resolution. How do these tasks contribute to building knowledge from unstructured text? Illustrate with examples. [8 Marks]

These three tasks work **together as a pipeline** to convert raw unstructured text into structured, machine-readable knowledge.

Simple analogy: Think of building a family tree from old handwritten letters.

- **Entity Extraction** = identify all the people and places mentioned
- **Relation Extraction** = figure out who is whose father, mother, sibling
- **Coreference Resolution** = realise that "grandfather", "the old man", and "he" all refer to the same person

Without all three working together, you cannot build a complete, accurate family tree.



Entity Extraction

Automatically identifies and classifies **meaningful pieces of information** (entities) from text — persons, organisations, locations, dates, amounts.

"Elon Musk founded Tesla in California in 2003." → Elon Musk **[PERSON]**, Tesla **[ORG]**, California **[LOC]**, 2003 **[DATE]**

Relation Extraction

Finds the **semantic relationship between pairs of entities** already identified by entity extraction. Output is a knowledge triple — (Subject, Relation, Object).

→ (Elon Musk, **founded**, Tesla) → (Tesla, **located_in**, California) → (Tesla, **founded_in**, 2003)

Coreference Resolution

Groups all words and phrases in text that **refer to the same real-world entity** into coreference chains — ensuring pronouns and alternate names are linked to their correct entity.

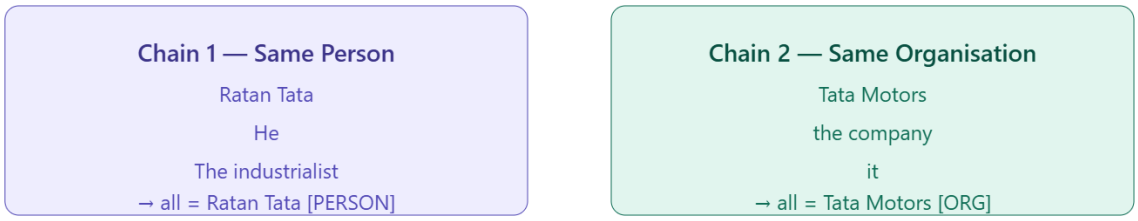
*"Elon Musk founded Tesla. **He** runs **the company** from California."* → He = Elon Musk → the company = Tesla

Complete Worked Example — All Three Together

Input paragraph:

"Ratan Tata built Tata Motors in Mumbai. He invested Rs 100 crore in the company. The industrialist said it would change India's automobile industry."

Step 1 — Coreference Resolution first:



Resolved paragraph:

*"Ratan Tata built Tata Motors in Mumbai. **[Ratan Tata]** invested Rs 100 crore in **[Tata Motors]**. **[Ratan Tata]** said **[Tata Motors]** would change India's automobile industry."*

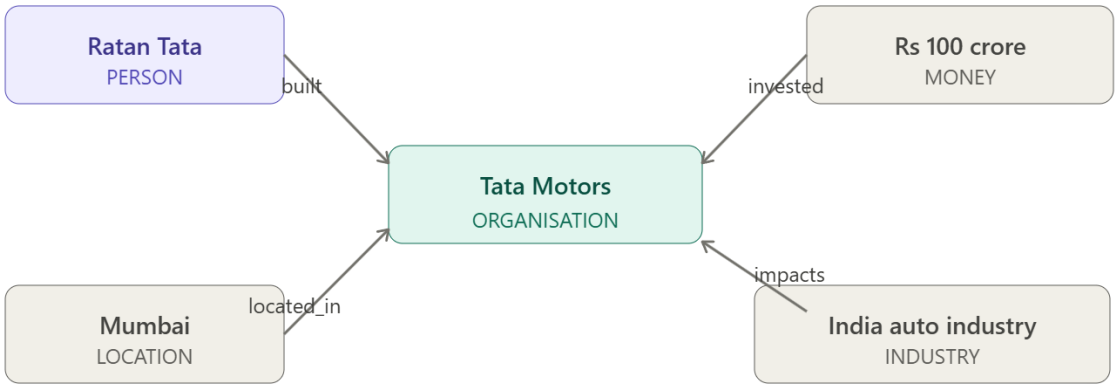
Step 2 — Entity Extraction:

Entity	Type
Ratan Tata	PERSON
Tata Motors	ORGANISATION
Mumbai	LOCATION

Entity	Type
Rs 100 crore	MONEY
India	LOCATION

Step 3 — Relation Extraction:

Triple	Meaning
(Ratan Tata, **built**, Tata Motors)	Ratan Tata founded the company
(Tata Motors, **located_in**, Mumbai)	Company is in Mumbai
(Ratan Tata, **invested**, Rs 100 crore)	Investment amount
(Tata Motors, **impacts**, India automobile industry)	Business impact



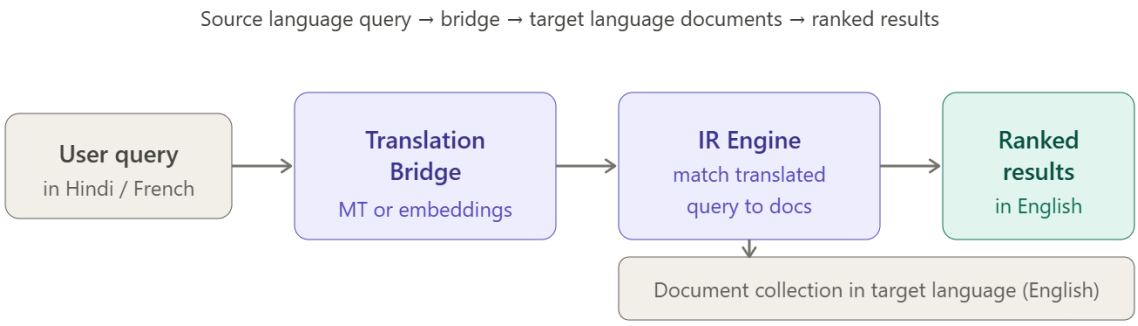
9) What is Cross-Lingual Information Retrieval and how is it used in NLP? Provide an example. [6 Marks] ★★★ OR Explain Cross Lingual Information Retrieval with example. [8 Marks] OR Define Cross-Lingual Information Retrieval (CLIR) and discuss the challenges involved in retrieving information from languages different from the query language. How do machine translation techniques assist in CLIR? [9 Marks] ★ OR What is Cross-Lingual Information Retrieval (CLIR)? Discuss the challenges of retrieving information across languages, and what techniques/models are commonly used to address these challenges? [8 Marks]

Cross-Lingual Information Retrieval (CLIR) is a type of Information Retrieval where the **query is written in one language** but the **documents to be retrieved are in a different language.**

Simple analogy: You are an Indian student who types a question in **Hindi** — "भारत का सबसे बड़ा बांध कौन सा है?" — and the system finds and returns relevant Wikipedia articles written in **English** about the largest dam in India. You searched in Hindi, the answer came from English documents. That is CLIR in action.

Normal IR: Query in English → Documents in English **CLIR:** Query in Hindi → Documents in English (or any other language)

How CLIR Works — Basic Architecture



Detailed Real-World Example

Scenario:

A Marathi-speaking researcher in Pune wants to find research papers about climate change. She types her query in Marathi: **"हवामान बदलाचे परिणाम"** (meaning: "Effects of climate change") Most research papers are written in **English.**

How CLIR handles this:

Step	What happens
Step 1	User types query in Marathi
Step 2	CLIR system translates query to English: "Effects of climate change"
Step 3	Translated query is matched against English document index
Step 4	TF-IDF and cosine similarity rank the most relevant papers
Step 5	Top results returned — optionally translated back to Marathi

Without CLIR, the Marathi researcher would find zero results because no English document contains Marathi words.

Challenges of CLIR

Challenge 1 — Translation Ambiguity

A word in one language may have multiple meanings when translated. Wrong translation leads to wrong documents being retrieved.

Hindi "कल" can mean **"yesterday"** OR **"tomorrow"** depending on context. If translated wrongly, completely irrelevant documents are returned.

Challenge 2 — Named Entity Translation

Person names, city names, and organisation names often do not translate directly. They may be spelled differently across languages.

"मुंबई" (Marathi/Hindi) = "Mumbai" = "Bombay" (all same city) A system that only knows "Mumbai" will miss documents that use "Bombay".

Challenge 3 — Morphological Differences

Different languages have very different word structures. In Hindi, a single word can carry the meaning of an entire English phrase.

English: "in the hospital" = 3 words Hindi: "अस्पताल में" = 2 words (different structure)

Challenge 4 — Language Resource Imbalance

Some languages have very little training data available for building translation models. This leads to poor translation quality for low-resource languages like Marathi, Swahili, or Nepali.

Challenge 5 — Script and Encoding Differences

Different languages use different scripts — Devanagari, Arabic, Chinese, Latin. Handling multiple scripts requires special encoding and tokenisation.

Challenge 6 — Cultural Context and Idioms

Idiomatic expressions do not translate literally.

Hindi: "आसमान से तारे तोड़ना" literally = "pluck stars from sky" but actually means "do the impossible" A literal translation gives completely wrong search results.